TGCAE: Temporal Graph Convolutional Autoencoder for Robust Anomaly Detection in Object-Centric Process Mining

Jun Cao
Anhui University of Science and Technology
Huan Fang
2003122@aust.edu.cn

Anhui University of Science and Technology
Cong Liu
University of Lisbon

Research Article

Keywords: Object-centric Event Log, Process Mining, Anomaly Detection, Temporal Graph Convolutional Autoencoder

Posted Date: March 6th, 2026

DOI: https://doi.org/10.21203/rs.3.rs-8816450/v1

Additional Declarations: No competing interests reported.

TGCAE: Temporal Graph Convolutional Autoencoder for Robust Anomaly Detection in Object-Centric Process Mining

Jun Cao[1], Huan Fang[1*], Cong Liu[2]

[1*]College of Mathematics and Big Data, Anhui University of Science and Technology, 168 Taifeng Street, Huainan, 232001, Anhui, China.
[2]NOVA Information Management School, Nova University of Lisbon, Lisbon, 1070-312, Portugal.

*Corresponding author(s). E-mail(s): 2003122@aust.edu.cn;
Contributing authors: 2024201455@aust.edu.cn; cliu@novaims.unl.pt;

Abstract

With the deep integration of machine learning and cybernetics technologies in business process management (BPM), robust anomaly detection in object-centric process mining is crucial for enhancing operational efficiency and quality within enterprises.Existing anomaly detection approaches predominantly rely on flattened event logs, which overlook multi-object interactions and their complex dependency relationships, thereby limiting the capability to model intricate business processes. In contrast, Object-Centric Event Logs (OCELs) associate events with multiple objects, enabling the modeling of multi-object interactions and complex dependencies inherent in real-world processes. However, current Graph Neural Network (GNN)-based anomaly detection methods primarily focus on structural features while neglecting the critical temporal dynamics of process execution. To bridge this gap, this paper proposes a Temporal Graph Convolutional AutoEncoder (TGCAE) for robust anomaly detection in object-centric process mining. TGCAE effectively captures structural dependencies between process execution instances via Graph Convolutional Networks (GCNs), and integrates a dedicated temporal encoder with an adaptive gating mechanism to model complex temporal patterns. For anomaly detection, a data-driven Interquartile Range (IQR) method is employed to adaptively determine decision thresholds, enabling accurate identification of three typical anomaly types in event logs: attribute manipulation, timestamp perturbation, and activity insertion. Comprehensive experiments on both real and synthetic datasets demonstrate that TGCAE significantly outperforms traditional Autoencoder (AE), Long Short-Term Memory Autoencoder (LSTMAE), and Graph Convolutional Autoencoder (GCNAE) baselines, validating the efficacy of spatiotemporal information fusion for robust anomaly detection.

Keywords: Object-centric Event Log, Process Mining, Anomaly Detection, Temporal Graph Convolutional Autoencoder

1 Introduction

Business Process Management (BPM) constitutes a core component of information-based management in modern enterprises. With the widespread

application of information systems, enterprises have accumulated massive volumes of process execution log data [1]. Traditional flattened event logs adopt a case-centric perspective, where each event is associated with only a single case identifier. However, real-world business processes typically involve intricate interactions among multiple objects. For example, in an e-commerce order processing workflow, a payment event may be linked to three objects (i.e., order, user, and logistics), and an after-sales refund event may relate to two cases (i.e., the original order and a new work order). Such cross-object and cross-case event relationships are simplified as isolated sequential nodes in flattened logs, rendering traditional methods incapable of capturing abnormal patterns hidden in object interactions. Object-Centric Event Logs (OCELs) address the limitations of flattened logs by allowing each event to be associated with multiple objects, thus reflecting the complexity of business processes more realistically.

In practical operations, various abnormal situations inevitably occur during process execution due to system malfunctions, human errors, and other factors. Failure to detect and handle these anomalies in a timely manner can lead to severe economic losses and operational risks. Conventional anomaly detection methods for business processes are primarily based on rules [2, 3] or statistical models [4], which require domain experts to define abnormal patterns and therefore struggle to adapt to complex and dynamic business scenarios. In recent years, the rapid development of machine learning and deep learning technologies has provided new insights for anomaly detection. In particular, Graph Neural Networks (GNNs) [5, 6] can effectively model complex relationships between process execution instances and have demonstrated excellent performance in anomaly detection tasks. Nevertheless, existing GNN-based anomaly detection methods for business processes mainly focus on structural or attribute features of processes, identifying anomalies by learning structural representations of graphs of normal processes [7, 8, 9]. However, business processes exhibit prominent temporal characteristics in addition to structural and attribute features. Temporal information such as the occurrence time of events, the execution sequence of activities, and time intervals in processes is of great significance for anomaly detection. Neglecting temporal information can degrade the accuracy of anomaly detection and hinder efficient identification of abnormal patterns [10]. Therefore, how to effectively fuse structural and temporal information within the GNN framework has become a key issue to improve the anomaly detection capability of business processes.

To address the aforementioned problems, this paper proposes a Temporal Graph Convolutional AutoEncoder (TGCAE) model specifically designed for anomaly detection on OCELs. TGCAE captures the interaction relationships and structural dependencies among objects via GNNs, while introducing a temporal encoder and an adaptive gating mechanism to model the temporal patterns of events. Aiming at the multi-object characteristic of OCELs, an object type-aware graph construction method and a weighted anomaly scoring strategy are designed. Finally, extensive experiments are conducted on two types of public OCELs datasets (i.e., real-world and synthetic datasets). By injecting three types of anomalies with different proportions into the event logs, this paper systematically evaluates the performance of the TGCAE model. The experimental results demonstrate that TGCAE achieves excellent and stable performance in the detection of anomalies in event activity and attribute anomalies. Compared with baseline models (AE [11, 12], LSTMAE [11, 13, 14] and GCNAE [8]), TGCAE produces significant improvements in core metrics such as Area Under the Precision-Recall Curve (AUC-PR) and F1-score, and exhibits stronger robustness and adaptability under high anomaly contamination rates. For temporal anomaly detection, TGCAE achieves a performance breakthrough compared with typical GNN models, providing an efficient and feasible solution for complex anomaly detection on OCELs. This also verifies the effectiveness of the temporal-structural fusion framework in OCELs anomaly detection.

2 Preliminaries

This section briefly introduces the basic concepts related to OCEL and business process anomaly detection.

2.1 Object-Centric Event Logs

The basic concepts of object-centric business process anomaly detection are presented below, including domain, events and attributes, and object-centric event logs.

Definition 1 (Domain [15]) In an object-centric event log, it contains at least events, activities, timestamps, and association relationships with one or more objects. Herein, E denotes the domain of events, A_{act} denotes the domain of activities, A_{att} denotes the domain of attribute names, A_{val} denotes the domain of attribute values, denotes the domain of object instances, O_{type} denotes the domain of object types, and T denotes the domain of timestamps.O_{obj} denotes the domain of object instances, O_{type} denotes the domain of object types, and T denotes the domain of timestamps.

Definition 2 (Time and Attribute [15]) Let an event $e \in E$ be a tuple, denoted as$e = (a, t, o)$, where $a \in A_{act}$ represents the activity corresponding to the event, $t \in T$ represents the timestamp corresponding to the event, and $o \in \pi_{obj}(e)$ represents the object instance associated with the event. Define $\pi_{attr} : E \times A_{att} \to A_{val}$ as the mapping function of event attributes, where $\pi_{attr}(e, a)$ is the specific value of event $e \in E$ for attribute $a \subseteq A_{att}$.

Definition 3 (OCEL [15]) OCEL is a specialized event log format, as shown in Eq.(1).

$$OCEL = \big(E, T, O_{\text{obj}}, O_{\text{type}}, A_{\text{act}}, A_{\text{att}}, A_{\text{val}}, \pi_{\text{time}}, \pi_{\text{obj}}, \pi_{\text{act}}, \pi_{\text{attr}}\big) \tag{1}$$

Here, π_{time} denotes $E \to T$, i.e. the mapping that associates each event with a timestamp; π_{obj} denotes $E \to O_{obj}$, associating each event with an object identifier; π_{otyp} denotes $O_{obj} \to O_{type}$, indicating each object possesses a unique object type; π_{act} denotes $E \to A_{act}$, mapping each event to its associated activity; π_{attr} denotes $E \times A_{att} \to A_{act}$, a partial function mapping events and attributes to their attribute values.

Definition 4 (Object-Centric Process Instance [8]) Let L be an object-centric event log. For each object $o_i \in L$, its corresponding temporal event trace is denoted as $\pi_{trace}(o_i) = \langle e_1, ..., e_n \rangle$. Then, the object-centric process instance $P = \big(E', D\big)$ is a directed graph, where nodes E' represent events, edges D represent the temporal dependency relationships of events, and the graph corresponds to a set of traces directly or indirectly associated through one or more common events.

2.2 Object-Centric Business Process Anomaly Detection

Object-centric business process anomaly detection methods take business objects (such as orders, customers, products, etc.) as the core analysis unit, integrate multi-source data related to objects (including attributes, lifecycle states, and interaction records with other objects, etc.), and realize fine-grained anomaly identification in business processes. Unlike traditional methods that focus only on the sequence of process activities, this method emphasizes the intrinsic correlation between object states and process execution traces.

Definition 5 (Normal Events and Anomalous Events) Given an object-centric event log L, an anomaly detection function $f : E \to \{0, 1\}$ divides E into two categories: E_n and E_a, where $E_n = \{e \in E | f(e) = 0\}$ is the set of normal events and $E_a = \{e \in E | f(e) = 1\}$ is the set of anomalous events.

Definition 6 (Attribute Swap) Let P be the set of process instances. For any process instance $p \in P$, denote V_p as the set of all events in this process instance. For any event $i \in V_p$, its attribute vector is $x_i \in \mathbb{R}^d$ (where d is the dimension of the attribute vector). Select the event j from V_p that maximizes the Euclidean distance $\|x_i - x_k\|_2$, i.e., $j = \text{argmax}_{k \in V_p} \|x_i - x_k\|_2$, and replace the attribute vector of the event i with x_j.

Definition 7 (Timestamp Shift) Let O be the set of objects in the object-centric event log. For any object $o \in O$, denote ε_o as the set of all events associated with object o. For any candidate event $i \in \varepsilon_o$, randomly sample an event j from $\varepsilon_o \backslash \{i\}$ (where $\backslash$ denotes the set difference operator). Let the original timestamp of the event j bet$_j$ and offset t_j to obtain a new timestamp t'_j, which satisfies $t'_j \in [t_j \cdot (1 - \Delta t_1), t_j \cdot (1 + \Delta t_2)]$, where Δt_1 is the downward disturbance ratio of the timestamp and Δt_2 is the upward disturbance ratio of the timestamp.

Definition 8 (Random Activity Injection) Let p be the original process instance, p' be the target process

instance, and $A(p)$ and $A(p')$ be the sets of activity types of p and p', respectively, with $A(p) \cap A(p') \neq \emptyset$. $X(p')$ is the set of attribute values for all events in p'. Select an activity type $a \in A(p') \setminus A(p)$, generate a new event $e = (a, v_1, v_2, \ldots, v_k)$, and inject the event into p as $p \Leftarrow p \oplus e$, where $v_i \sim X(p')_i$ is a random sample, $X(p')_i$ denotes the value domain of the i-th type of attribute in $X(p')$, and $\oplus$ denotes the temporal insertion operation.

3 Related Work

Current business process anomaly detection methods mainly target two types of event logs: flattened event logs and object-centric event logs. Traditional anomaly detection methods are based on "flattened" event logs, where events are characterized by a single case identifier, process instances are predefined as strictly ordered event sequences, and each event is associated only with one case [16]. This modeling approach is difficult to characterize the complex features of multi-case interactions and multi-object associations in real-world business scenarios. Therefore, research on anomaly detection for object-centric event logs has important practical significance and research value. Such methods take object instances as the core carrier; each event in the log is associated with multiple object identifiers, and simultaneously records the association relationships and state transition processes between objects, which can more accurately restore the operation logic of complex business processes. In line with traditional event logs, each event in an object-centric event log also contains activity type, timestamp, and other attribute information.

3.1 Anomaly Detection for Flattened Event Logs

For traditional flattened event logs, machine learning-based methods are often used for anomaly detection. Kang et al. [17] utilized the Local Outlier Factor (LOF) based on K Nearest Neighbor Imputation (KNNI) to detect anomalies. Sureka [18] calculated the Normalized Longest Common Subsequence (NLCS) between traces and applied the K Nearest Neighbors (KNN) algorithm for anomaly detection. Folino et al. [19] extracted structural patterns and used an anomaly-aware clustering algorithm for anomaly detection. Tavares et al. [20] regarded activities and traces as words and sentences, respectively, used the word2vec [21] encoding method to encode traces into vectors, and then adopted the random forest algorithm to classify traces into normal and anomalous. In contrast, Junior [22] used a support vector machine to classify traces as normal or anomalous. Later, some researchers proposed reconstruction-based methods for anomaly detection. These methods first train a neural network model that can reconstruct normal behaviors, and then detect anomalies based on reconstruction errors. Nguyen [11], Krajšić [12], et al. used autoencoders for anomaly detection. Guan et al. [23] fused multi-view information and an attention mechanism to improve the performance of multi-view business process anomaly detection. Since autoencoder-derived models often face the risk of overfitting, Nolle et al. [[24, 25] applied the denoising of the autoencoders, introducing additive Gaussian noise to the input vectors, and achieved good results. Subsequently, Evermann [26], Tax [27], et al. used LSTM [28] to predict the next event in the sequence of events, and Nolle et al. [13, 29] proposed BINet based on Gated Recurrent Unit (GRU) to predict the next event in the sequence of events. These methods detect anomalies by predicting the probability of the next event attribute.

However, a business process is a set of structured activities with intrinsic relationships. These methods do not consider this information, which may limit the improvement of anomaly detection performance. Therefore, graph-based methods are widely used, which rely on graphs modeled by relationships between activities to detect anomalies. Ebrahim [30], Sarno [3], et al. adopted conformance checking technology [31] to evaluate the conformance between the event log and the corresponding process model by using a process model provided by process mining technology or generated from the log. Vitale et al. [32] performed process mining feature extraction based on alignment-based conformance checking and integrated it into a flexible and interpretable framework to provide an adaptive solution for control-flow anomaly detection. Böhmer et al. [33] proposed a method based on probabilistic graph models to map traces to likelihood graphs, using the probability of attribute values to determine whether they are anomalous. Rahmawati et al.

[34] used the Hidden Markov Model (HMM) to analyze the event logs. Linn et al. [35] adopted three sequence analysis techniques based on windows, Markov models, and hidden Markov models to detect anomalies in business processes.

In addition, modeling business processes as graphs and using Graph Neural Networks (GNNs) can obtain graph embeddings of business processes, and then detect anomalies based on reconstruction errors. For example, the Graph Autoencoders (GAE) proposed by Huo et al. [36] used Edge-Conditioned Convolution (ECC) to obtain better graph encoding to improve anomaly detection performance. Guan et al. [9] proposed a multi-graph anomaly detection framework GAMA, modeling each attribute in the business process trajectory as an independent graph, and calculating anomaly scores based on reconstruction errors to achieve detection. In recent years, the wide application of large language models has provided new ideas and technical support for anomaly detection. Guan et al. [37] proposed the DABL method, generating normal trajectories through real process models, simulating ordering anomalies and exclusion anomalies, and fine-tuning the Llama 2 model based on QLoRA to realize semantic anomaly detection and natural language explanation.

3.2 Anomaly Detection for Object-Centric Event Logs

Object-centric anomaly detection is a technique that focuses on understanding the behavior of individual objects in a system or dataset to effectively detect anomalies. However, it faces problems such as scalability anomalies, dynamic attribute change anomalies, and insufficient support for O2O relationships [38], which affect the effectiveness of OCELs in comprehensive process analysis [39]. Berti et al. [40] proposed various object-centric anomaly detection methods. By analyzing the lifecycle and interactions of objects, anomalies are detected based on deviations from expected patterns or behaviors. They also explored the role of large language models and feature propagation, providing multi-dimensional solutions for anomaly detection in complex object interaction scenarios. Although object-centric process mining has made progress in capturing complex and multi-dimensional event log data, anomaly detection remains challenging, especially when dealing with data involving multiple entities. Adams et al. [41] generalized the concept of "case" in traditional process mining to "process execution" oriented to object-centricity, proposed two techniques: connected component extraction and dominant type extraction, combined graph isomorphism to identify equivalent process executions, and visualized object-centric variants. Amin et al. [42] proposed a method that integrates object-centric event logs into Event Knowledge Graphs (EKG) and uses graph-based techniques for anomaly detection, which successfully achieved effective detection of structural anomalies and interaction-related anomalies in OCEL. Niro [8] proposed an unsupervised anomaly detection framework based on Graph Convolutional AutoEncoder (GCNAE), which does not require prior process models, contamination rate information, or clean training sets, providing a new solution for event-level anomaly detection of object-centric processes.

In summary, business process anomaly detection methods have evolved from flattened event logs to object-centric event logs. For flattened event logs, machine learning-based methods focus on statistical feature extraction but struggle to capture the structural characteristics of processes; deep learning-based reconstruction methods detect anomalies through reconstruction errors while ignoring the intrinsic relationships between activities; GNN-based methods model process relationships using graph structures, which, to a certain extent, make up for the deficiencies of the previous two types of methods. However, flattened logs cannot effectively characterize complex scenarios such as multi-object interactions and cross-case associations in real-world businesses. For object-centric event logs, existing methods treat timestamps as ordinary attribute features and cannot explicitly model the temporal dependencies of events, leading to limited anomaly detection capabilities [43]. Therefore, how to effectively integrate the structural information of object interactions and the temporal information of event occurrences in object-centric scenarios to construct a more robust and comprehensive anomaly detection model remains a difficult problem in the field of business process mining.

4 Methodology

The Temporal Graph Convolutional AutoEncoder (TGCAE) model proposed in this paper is designed to address the anomaly detection problem in Object-Centric Event Logs (OCELs), enabling the joint detection of structural anomalies, attribute anomalies and temporal anomalies. TGCAE converts timestamps and residual graph information into temporal embeddings and structural embeddings via a temporal encoder and a structural encoder, respectively. A gating mechanism is then employed to adaptively fuse the structural and temporal features of nodes, allowing the model to capture both graph structural information and temporal dynamics simultaneously. This design effectively extracts the topological characteristics of graph structures and the temporal dependencies of node attributes. Specifically, the temporal encoder transforms discrete timestamps into continuous vector representations, providing temporal information input for subsequent processing. Graph Convolutional Networks (GCN) or Graph Isomorphism Networks (GIN) leverage these feature vectors and inter-node relationships to learn dynamic node representations and capture complex evolutionary patterns of graph structures, followed by the adaptive fusion of spatiotemporal information through the gating mechanism. Through this approach, TGCAE achieves efficient modeling of complex anomaly patterns in the encoded graph, and can process graph structural information and time series concurrently, thereby improving the accuracy and robustness of anomaly detection.

4.1 Reconstruction of Process Instances from Event Logs and Graph Encoding Embedding

Process instance reconstruction refers to, in the modeling process of event log L, first reconstructing each process instance in the original log into an object-centric directed graph $G = (V, E)$. This reconstruction process relies on the OCPA framework, constructing node set V and edge set E by parsing activity sequences and object association relationships in the event log. In this paper, node $v \in V$ corresponds to each activity instance in the process, and its attributes include activity type $\pi_{act}(v)$, timestamp t_v, and related object identifiers; edge $e \in E$ represents the direct successor relationship between activities, i.e., control flow constraints. Specifically, for each process instance P_i, extract the event_id of all its nodes, and construct an adjacency matrix $\mathbf{A}_L$ based on these nodes, where $A_L[i][j] = 1$ indicates that there is an edge between node i and node j, otherwise $A_L[i][j] = 0$.

For example, from the simple object-centric event log L in Figure 1, the reconstructed process instances P_1 and P_2 are obtained. Among them, P_1 consists of the set of traces $\pi_{trace}(x_1) \cup \pi_{trace}(x_3) \cup \pi_{trace}(y_3) = \langle e_1, e_3, e_8 \rangle \cup \langle e_3, e_8 \rangle \cup \langle e_3, e_6 \rangle$, and these traces are associated through events e_3 and e_8; P_2 consists of the set of traces $\pi_{trace}(x_2) \cup \pi_{trace}(y_1) \cup \pi_{trace}(y_2) = \langle e_2, e_5, e_7 \rangle \cup \langle e_2, e_4, e_7 \rangle \cup \langle e_2, e_7 \rangle$, and these traces are associated through events e_2, e_4, and e_7.

Subsequently, during input graph encoding, we combine all process instances into a single graph G, which is composed of a set of disconnected subgraphs (i.e., each process instance). This approach avoids the need for padding or other transformations on each instance graph, thereby preserving the integrity of the original graph structure. For example, in the example shown in Figure 1, the input graph G_L corresponds to the set of two subgraphs P_1 and P_2, i.e., $G_L = \{P_1, P_2\}$. Each subgraphP_i represents an independent process instance, and its node set V_i and edge set E_i correspond to the activity instances and their control flow relationships in this instance, respectively. We then further encode each input graph G_L into a structural feature matrix $X \in \mathbb{R}^{|V| \times d}$.

Definition 9 (Structural Feature Matrix,X) Given an object-centric event log L and its input graph $G = (V, E)$, we map each node $v \in V$ to a structural feature vector $x_v \in \mathbb{R}^d$, which corresponds to the event activity type $\pi_{act}(v)$ and other original attributes $\pi_{att}(v)$ after removing the timestamp column. Among them, categorical features are encoded using one-hot encoding, and numerical attributes are standardized. Therefore, d is equal to the sum of the number of activity types in A_L, the number of categorical attribute values in A_L, and the number of numerical attributes. Finally, the feature vectors of all nodes are composed into a matrix $X \in \mathbb{R}^{|V| \times d}$, where $X = [x_1, x_2, \cdots, x_{|V|}]^T$. The encoded form of the input graph G is still $G = (A, X)$. For the object-centric event log L in Figure 1, its adjacency matrix

A_L and feature matrix X_L are shown in Eqs. (2) and (3).

$$\boldsymbol{A}_L = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \tag{2}$$

$$\boldsymbol{X}_L = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0.25 & 0.47 & \cdots \\ 0 & 1 & 0 & 0 & 0 & 0.54 & 0.21 & \cdots \\ 1 & 0 & 0 & 0 & 0 & 0.36 & 0.86 & \cdots \\ 0 & 0 & 1 & 0 & 0 & 0.42 & 0.34 & \cdots \\ 0 & 1 & 0 & 0 & 0 & 0.72 & 0.56 & \cdots \\ 0 & 0 & 0 & 1 & 0 & 0.14 & 0.26 & \cdots \\ 0 & 0 & 0 & 0 & 1 & 0.65 & 0.49 & \cdots \\ 0 & 0 & 1 & 0 & 0 & 0.26 & 0.33 & \cdots \end{bmatrix} \tag{3}$$

After completing the graph encoding process, we enter the key embedding phase, which aims to map the original node features into a low-dimensional semantic space to capture the intrinsic structural patterns of the data.

Definition 10 (Structural Embedding) Structural embedding $H_{struct} \in \mathbb{R}^{|V| \times d_h}$ is a node representation obtained by encoding the structural feature matrix through a graph convolutional network, which is calculated as shown in Eq. (4).

$$H_{struct} = StructureEncoder(X, A) \tag{4}$$

where $X \in \mathbb{R}^{|V| \times k}$ is the structural feature matrix, A is the adjacency matrix, and k is the dimension of structural features. This embedding aggregates neighbor node information through graph convolution operations, effectively capturing the interaction relationships and structural dependency patterns between objects, and will be fused with temporal embedding subsequently.

Definition 11 (Temporal Embedding) Temporal embedding $H_{time} \in \mathbb{R}^{|V| \times d_h}$ is a high-dimensional representation obtained by mapping the standardized timestamp vector through a temporal encoder, which is calculated as shown in Eq. (5).

$$H_{time} = TemporalEncoder(T) \tag{5}$$

where $|V|$ denotes the number of nodes, d_h is the hidden layer dimension, and the timestamp vector $T \in \mathbb{R}^{|V| \times 1}$ is standardized as the input of the temporal encoder. This embedding maps one-dimensional timestamps to a d_h-dimensional space through a temporal encoder, learns the complex pattern features of time series, and maintains the output within a bounded range, which is conducive to subsequent fusion with structural embedding.

4.2 TGCAE Architecture

The Temporal Graph Convolutional AutoEncoder is a multi-anomaly detection model designed for OCELs, as shown in Fig. 1. It mainly consists of five core modules: structural encoder, temporal encoder, adaptive gating fusion mechanism, decoder, and anomaly scoring. In the data preprocessing phase, the original object-centric event log is reconstructed into a series of process instance graphs $G = (A, X)$, where the adjacency matrix A characterizes the topological relationships between nodes, and the feature matrix X encodes node attributes. Meanwhile, the timestamp T of each event is standardized as an independent temporal input. This "separation and fusion" paradigm ensures the decoupling and coordination of temporal information and structural information in subsequent processing.

The encoder module includes two parallel branches: a structural encoder and a temporal encoder. The former uses graph neural networks to extract structural embeddings, and the latter generates temporal embeddings through a multilayer fully connected network. Subsequently, these two embeddings are fed into an adaptive gating fusion mechanism, which can dynamically learn a weight for each node to determine its dependence on structural and temporal information, realizing adaptive information fusion, thereby improving the robustness of overall anomaly detection.

Finally, the decoder module receives the fused embeddings and attempts to reconstruct the original features. The anomaly score of the model is calculated based on the reconstruction error, and differential weights can be assigned according to different attribute dimensions to more accurately identify specific types of anomalous events.

This architecture introduces three core innovations:

(1) A dedicated temporal encoder with a fully connected three-layer network to extract complex temporal patterns from standardized timestamps;

(2) An adaptive gating fusion mechanism to dynamically balance the contributions of structural and temporal information at the node level;

(3) A weighted reconstruction error strategy combined with an IQR-based threshold determination method to achieve robust anomaly scoring in unknown contamination scenarios.

4.2.1 Structural Encoder

To accurately capture the structural relationships between objects, a structural encoder is designed in the model, which adopts Graph Convolutional Networks (GCN) or Graph Isomorphism Networks (GIN) as the basic architecture. Given a process graph $G = (A, X)$, where A is the adjacency matrix of the graph and $X \in \mathbb{R}^{|V| \times k}$ is the node feature matrix, the structural encoder learns the structural embedding representation of the nodes through multi-layer graph convolution operations, as shown in Eq. (6).

$$H_{struct}^{(l+1)} = \sigma\left(\widetilde{D}^{-1/2}\widetilde{A}\widetilde{D}^{-1/2}H_{struct}^{(l)}W^{(l)}\right) \quad (6)$$

where $\widetilde{A} = A + I$ is the adjacency matrix with self-loops added, $\widetilde{D}$ is the corresponding degree matrix, $W^{(l)}$ is the weight matrix of the l-th layer, σ is the activation function, and $H_{struct}^{(0)} = X$ is the initial node feature. The output of the structural encoder $H_{struct} \in \mathbb{R}^{|V| \times d_h}$ contains the structural information of nodes, where d_h is the hidden dimension.

4.2.2 Temporal Encoder

The temporal encoder is one of the core innovations of TGCAE, specifically designed to process standardized timestamps and extract temporal features. Different from the way GCNAE [8] treats timestamps as ordinary feature dimensions, the temporal encoder adopts a deep nonlinear network architecture, which can learn complex nonlinear transformations of timestamps and capture patterns at different time scales. The temporal encoder consists of a fully connected three-layer network, as shown in Eq. (7).

$$\begin{aligned} H_{\text{time}} = \tanh\big(&\text{Linear}_{d_h}(\text{Dropout}(\text{ReLU}(\text{Linear}_{d_h} \\ &\left(\text{Dropout}\left(\text{ReLU}\left(\text{Linear}_{\frac{d_h}{2}}(T)\right)\right)\right)\right)\right)\big) \end{aligned} \quad (7)$$

where $T \in \mathbb{R}^{|V| \times 1}$ is the standardized timestamp vector, $H_{time} \in \mathbb{R}^{|V| \times d_h}$ is the learned temporal embedding, ReLU is the activation function, Dropout is the regularization function, and tanh is the output activation function.

4.2.3 Adaptive Gating Fusion Mechanism

Another core innovation of TGCAE is the adaptive gating fusion mechanism, which aims to dynamically balance the contributions of structural information and temporal information. Unlike the fixed-weight fusion method, this mechanism adaptively learns fusion weights for each node, allowing the model to flexibly adjust the information combination strategy according to the characteristics of the node. The specific calculation method is shown in Eq. (8).

$$\begin{aligned} H_{fusion} = (1 - g) \otimes H_{struct} + g \\ \otimes \text{MLP}_{fusion}\left([H_{struct}; H_{time}]\right) \end{aligned} \quad (8)$$

$$g = \sigma\left(\text{MLP}_{gate}\left(H_{struct}; H_{time}\right)\right) \quad (9)$$

where g is the gating weight, σ is the Sigmoid function, MLP_{fusion} is a fully connected 4-layer network used to generate fused features,$\cdot;\cdot$ denotes the vector concatenation operation and $\otimes$ denotes the multiplication of elements. MLP_{gate} in Eq. (9) is a fully connected five-layer network used to calculate the gating value.

This design allows the model to adaptively adjust the contribution ratio of structural and temporal information according to node characteristics, which not only retains the original structural information, but also introduces temporal information, making the model's detection ability for various anomalies more balanced.

4.2.4 Decoder

The decoder adopts a symmetric GCN or GIN structure with respect to the encoder, reconstructing the fused embedding H_{fusion} back to the

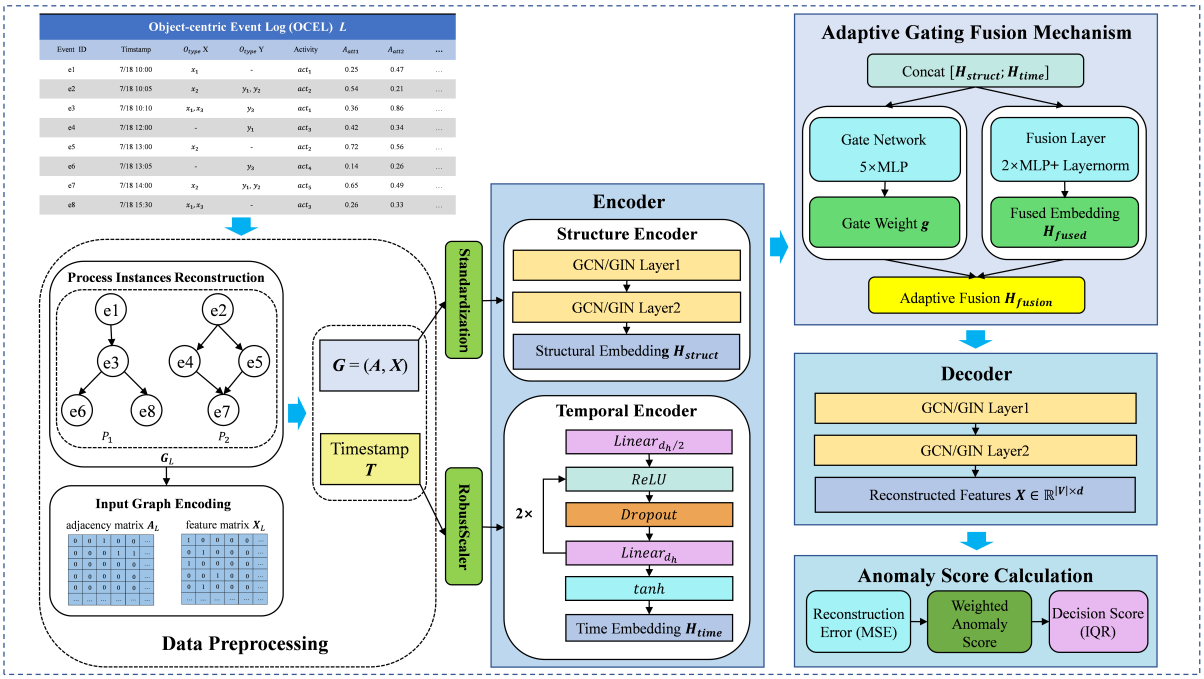

Fig. 1 TGCAE Framework

original feature space. The reconstruction error serves as the basis for anomaly scoring, and the calculation method is shown in Eq. (10).

$$\hat{X} = \mathrm{GCN}_{decoder}\left(H_{fusion}, A\right) \tag{10}$$

where $\hat{X} \in \mathbb{R}^{|V| \times d}$ is the reconstructed node feature matrix. The goal of the decoder is to minimize the difference between the original feature$X$ and the reconstructed feature $\hat{X}$, thus learning low-dimensional embeddings that can effectively represent normal patterns.

4.2.5 Model Training and Anomaly Scoring

The training goal of TGCAE is to minimize the reconstruction error, capturing the behavioral patterns of normal events through unsupervised learning. The loss function is defined as Eq. (11).

$$L = \frac{1}{|V|} \sum_{\nu \in V} s(\nu) \tag{11}$$

where $s(\nu)$ is the weighted anomaly score of nodeν. Considering that different attribute dimensions in OCEL correspond to different types of anomaly features, we adopt a weighted loss strategy, assigning differentiated weights to each attribute group. The calculation formula is shown in Eq.(12).

$$s(\nu) = \frac{\sum_{k=1}^{K} \omega_k \cdot \mathrm{MSE}_k(\nu)}{\sum_{k=1}^{K} \omega_k} \tag{12}$$

where $\mathrm{MSE}_k(\nu)$ is the mean square error reconstruction error of the attribute group k, and ω_k is the corresponding weight. The parameter configuration of this weighting mechanism is not based on prior assumptions but is optimized through systematic ablation experiments and multiple sets of comparative analyzes, under the trade-off of detection performance for different anomaly types. We evaluated the impact of various weight combinations on the overall performance of the model and finally selected a set of configurations that are optimal for global detection capabilities.

TGCAE adopts the Interquartile Range (IQR) method to adaptively generate anomaly detection thresholds to enhance the generalizability of the model in scenarios with unknown anomaly proportions. IQR is defined as the difference between the first quartile (Q_1) and the third quartile (Q_3) of the dataset, i.e., $\mathrm{IQR} = Q_3 - Q_1$. To determine the threshold for marking anomalies, we calculate

Q_3 and IQR of the set of anomaly scores generated by TGCAE, and set the threshold τ as shown in Eq. (13).

$$\tau = Q_3 + k \cdot \mathrm{IQR} \tag{13}$$

By setting the anomaly detection threshold, we can automatically mark events in the event log as normal or abnormal according to the anomaly score. Based on the characteristics of the normal distribution, k is set to 1.5 in this experiment, which makes the anomaly detection method more robust. However, this value can also be flexibly adjusted according to the actual data distribution characteristics or the model fitting effect to further optimize algorithm performance.

5 Experimental Analysis

To verify the effectiveness of the TGCAE model proposed in this paper for multi-type anomaly detection tasks in OCELs, this paper selects real and synthetic OCELs data to conduct multiple sets of experiments, and then compares and analyzes its performance with those of the AE [11, 12], LSTMAE [11, 13, 14], and GCNAE [8], respectively.

5.1 Experimental Preparation

5.1.1 Datasets

We evaluated the proposed method on both real-world and synthetic object-centric event logs. This dual verification method not only ensures the rigor and interpretability of the evaluation, but also tests the applicability of the method in actual business scenarios. Table 1 shows the summary data related to reconstructed process instances of each dataset under different contamination rates.

BPIC2017: This is a real-world business process log dataset associated with the loan application process of a Dutch financial institution. This event log includes all applications submitted through an online system in 2016 and their subsequent events, covering various complex business processes such as order management and payment processing. It involves two object types (application and offer) and thirteen attributes, and is characterized by multi-object dependencies with involvement of multiple entities including orders, payment requests and invoices. These features make it suitable for verifying the model's capability to handle multi-object interactions and complex event dependencies.

DS2: This is a synthetic event log that simulates an order management process with a large number of connected objects and high process variability [41]. This log contains three object types (item, order and package) and four attributes, designed to simulate the complex multi-object association scenarios in actual business processes and complement the real-world dataset for comprehensive model evaluation.

5.1.2 Anomaly Injection

A set of contamination rates (5%, 10%, 15%, 20%, 30%, 40%) was pre-defined to quantify the proportion of anomalous events relative to the total number of events in the log. For each contamination rate, three typical types of anomalies—attribute swap, timestamp shift, and random activity injection—were injected into the log with an equal proportion of allocation.

Attribute swap anomalies artificially introduce local attribute conflicts while preserving the sequential dependencies between events, effectively simulated by attribute inconsistency caused by data entry errors or sensor drift. This type of anomaly is particularly suitable for evaluating the sensitivity of models to local structural perturbations.

Timestamp shift anomalies disrupt the natural temporal order among events, thereby simulating the time sequence disorder induced by system latency, clock desynchronization, or artificial tampering. Such anomalies can effectively assess the stability and detection capacity of models in the face of temporal interference. In the experiments, we set $\Delta t_1 = \Delta t_2 = 0.05$.

Random activity injection anomalies mimic irrelevant activities or malicious injection behaviors that may occur in real-world scenarios, so as to evaluate the model's capability to detect unanticipated behaviors. This type of anomaly is especially applicable for assessing the generalizability and robustness of models under complex process scenarios.

[1] https://github.com/niro-a/DAEiOcBPvGNN/blob/main/ocel/BPI2017.csv

[2] https://github.com/niro-a/DAEiOcBPvGNN/blob/main/ocel/DS2.csv

Table 1 Datasets Statistics

Dataset	Original Events	Process Instances	Contam. Rate	Injected Anomalies	Final Events	Events per Process Instance (Max, Min, Avg)
BPIC2017[1]	393931	31509	5%	20022	400605	(44, 6, 12.7)
			10%	40704	407499	(45, 6, 12.9)
			15%	62043	414612	(45, 6, 13.2)
			20%	84036	421943	(46, 6, 13.4)
			30%	129996	437263	(48, 6, 13.9)
			40%	178581	453458	(50, 6, 14.4)
DS2[2]	22367	83	5%	1134	22745	(1413, 8, 274.0)
			10%	2310	23137	(1434, 8, 278.8)
			15%	3522	23541	(1467, 8, 283.6)
			20%	4770	23957	(1489, 8, 288.6)
			30%	7380	24827	(1552, 8, 299.1)
			40%	10137	25746	(1603, 8, 310.2)

This balanced anomaly injection strategy ensures the diversity of anomalous events across three dimensions (attribute, temporal sequence, and activity type), thus enabling a comprehensive evaluation of the model's detection performance for different types of anomaly. Furthermore, all anomaly injection operations were implemented under the assumption of preserving the structural characteristics of the original event log, which guaranties the comparability and reliability of the experimental results. Consequently, the robustness of models in the multi-modal noise scenarios commonly encountered in practical event log management can be fully evaluated.

5.2 Comparative Experiments

5.2.1 Evaluation Metrics

To comprehensively evaluate the performance of the proposed anomaly detection model, in this experiment a series of widely recognized evaluation metrics were adopted to assess the model's anomaly detection capability from multiple dimensions. For the evaluation of the model's overall performance, four core metrics were used to measure its general anomaly detection performance:

(1) $AUC-ROC$(Area Under the Receiver Operating Characteristic Curve): The ROC curve is obtained by plotting the relationship between the True Positive Rate (TPR) and the False Positive Rate (FPR). $AUC-ROC$ is the area under this curve, which is used to evaluate the classification performance of the model under different thresholds. Its value range is $[0, 1]$, and a higher value indicates better model performance. The calculation formulas are shown in Eqs. (14)∼(16).

$$TPR = \frac{TP}{TP + FN} \tag{14}$$

$$FPR = \frac{FP}{FP + TN} \tag{15}$$

$$AUC\text{-}ROC = \int_0^1 TPR(FPR)dFPR \tag{16}$$

where TP(True Positive) denotes correctly predicted anomalies, TN (True Negative) denotes correctly predicted normal events, FP (False Positive) denotes incorrectly predicted anomalies, and FN (False Negative) denotes incorrectly predicted normal events.

(2) $AUC-PR$ (Area Under the Precision-Recall Curve): The PR curve is obtained by plotting the relationship between Precision and Recall. $AUC-PR$ is the area under this curve, which is particularly suitable for the evaluation of imbalanced datasets and more sensitive to the anomaly detection capability. The calculation formula is shown in Eqs. (17)∼(19).

$$Precision = \frac{TP}{TP + FP} \tag{17}$$

$$Recall = \frac{TP}{TP + FN} \tag{18}$$

$$AUC\text{-}PR = \int_0^1 Precision(Recall) dRecall \quad (19)$$

(3) $F1-Score$: The $F1-score$ is the harmonic mean of $Precision$ and $Recall$, which comprehensively considers the balance between $Precision$ and $Recall$, and balances the trade-off between false positives and false negatives. The calculation formula is shown in Eq. (20).

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (20)$$

(4) $Recall@k$: The value of k is dynamically determined according to the contamination rate of the dataset (k = contamination rate $\times$ 100). For example, k=5 when the contamination rate is 5%, and $k = 10$ when the contamination rate is 10%, ensuring that the evaluation granularity matches the anomaly proportion.

For the evaluation of the specificity of the anomaly type, to deeply analyze the detection capacity of the model for different types of anomalies, the experiment further calculated the Recall@k value for each anomaly type ($T1$, $T2$, $T3$), where $T1$, $T2$, and $T3$ correspond to attribute swap, timestamp shift, and random activity injection anomalies, respectively. For each anomaly type Ti, its $Recall@k$ value was calculated separately. This method can reveal the performance differences of the model in detecting different anomaly patterns such as attribute anomalies, timestamp anomalies, and random event anomalies, providing a quantitative basis for accurately locating the advantages and shortcomings of the model.

Finally, through statistical analysis of the results of multiple runs, the experimental results are presented in the form of mean and standard deviation to evaluate the robustness and stability of each model. A smaller standard deviation indicates consistent performance of the model under different data splits or random initializations, ensuring the reliability of the experimental results. The calculation formulas for meanμ and standard deviation σ are shown in Eqs. (21) and (22).

$$\mu = \frac{\sum x_i}{n} \quad (21)$$

$$\sigma = \sqrt{\frac{\sum (x_i - \mu)^2}{n}} \quad (22)$$

Through the comprehensive analysis of these multi-dimensional evaluation metrics, the anomaly detection performance of the proposed model can be fully evaluated. It not only focuses on the overall detection accuracy but also deeply analyzes the detection capability for different types of anomalies, providing sufficient experimental evidence for the effectiveness of the model.

5.2.2 Comparative Methods

The performance of the TGCAE model was compared with existing traditional event log anomaly detection methods and current object-centric graph neural network-based anomaly detection methods. To enable comparison with traditional event log anomaly detection methods, the object-centric process instances were "flattened" into time-ordered sequences, where each sequence consists of the set of events contained in the corresponding process instance. Although this flattening strategy may still lead to divergence issues, it does not result in the deletion or duplication of events. As benchmark models, we implemented a standard Autoencoder (AE) similar to those in [11, 12], an LSTM Autoencoder (LSTMAE) similar to the recurrent neural network methods in [11, 13, 14], and an anomaly detection method based on Graph Convolutional Network Autoencoder (GCNAE) [8]. For the implementation of the three models, we used the hyperparameter settings provided in the corresponding literature [11, 13, 8], and Table 2 shows the hyperparameter settings of TGCAE. To ensure the reliability and statistical stability of the experimental results, the performance evaluation of all models was based on 5 independent repeated experiments, where each experiment initialized the model parameters with a different random seed. The final results are presented in the form of "mean $\pm$ standard deviation".

5.3 Results

5.3.1 Results on Real-World Logs

In the fields of process mining and graph representation learning, empirical evaluation based on

Table 2 Hyperparameter settings of TGCAE

Hyperparameter	Value
Hidden_dim	64
Encoder_layers	3
Decoder_layers	1
Learning_rate	0.004
Loss_func	F.mse_loss
Dropout_Rate	0.1
activation	F.relu

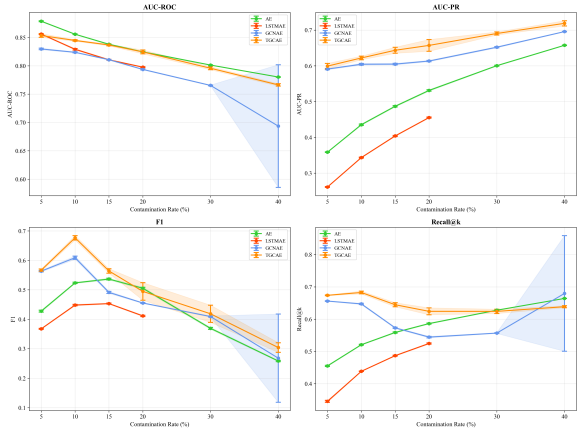

Fig. 2 Comparison of Main Performance Metrics Across Models in the Real Dataset

real-world datasets is an indispensable core pillar of rigorous scientific research, as it can truly reflect the inherent complexity, noise, and irregularities of actual systems. Referring to the anomaly simulation strategies in [11, 13, 14, 24], we injected different proportions of artificial anomalies into real event logs to simulate various special situations that may occur in real-world business processes. To evaluate the performance of different autoencoder models under different data contamination levels, we systematically analyzed the performance of the $AUC-ROC$, $AUC-PR$, $F1-score$, $Recall@k$, and $R@k-Ti$ metrics at each contamination rate, and the results are summarized in Table 3.

As shown in Table 3 and Fig. 2, TGCAE consistently exhibited excellent performance under different contamination rates, demonstrating its robustness and effectiveness in anomaly detection tasks.

At low contamination rates (5% and 10%), TGCAE achieved the highest $AUC-PR$, $F1-score$, and $Recall@k$ metrics. Meanwhile, its value $AUC-ROC$ showed extremely strong stability, with a mean value only slightly lower than that of the AE baseline model. This comprehensive performance indicates that TGCAE is significantly superior to the traditional AE, LSTMAE, and GCNAE models overall, and its advantages are more pronounced especially in precision-oriented metrics such as $AUC-PR$.

At moderate contamination rates (15% and 20%), TGCAE maintained its competitive advantage continuously. Its $AUC-PR$ and Recall@k metrics were significantly higher than those of the baseline models, while the $F1-score$ also remained at a high level. With the increase of contamination rate, the $AUC-PR$ curve of TGCAE was always above those of other models, and its growth rate was more stable, indicating that the model did not show obvious fluctuations in the recognition accuracy and recall capability of abnormal samples when dealing with medium-intensity noise. In addition, the robust performance of the $F1-score$ also reflects that TGCAE has achieved a better balance between precision and recall.

At high contamination rates (30% and 40%), although all models exhibited performance degradation—a predictable result of increased noise in training data—TGCAE still maintained the highest AUC-PR value and achieved stable or suboptimal performance on other metrics. It should be noted that LSTMAE encountered Out-of-Memory (OOM) errors at high contamination rates, which highlights the scalability limitations of sequence models in large-scale process mining datasets. In contrast, the graph-based architecture of TGCAE demonstrated better memory efficiency, enabling it to maintain stable performance even under extreme contamination conditions.

Combined with the type-specific recall data in Table 2 and Fig. 3, the balanced capability of TGCAE in various anomaly detection tasks can be analyzed. Although all graph-based models (TGCAE and GCNAE) achieved significant improvements in the performance of attribute anomaly detection and activity anomaly detection compared to traditional autoencoder models (AE and LSTMAE), it should be noted that TGCAE achieved substantial performance improvements over GCNAE in both of these anomaly detection tasks, and also showed significant improvements in

Table 3 Comparative Analysis of Model Metrics on the Real Dataset

Contam. Rate(k%)	Model	AUC-ROC (%)	AUC-PR (%)	F1 (%)	Recall@k (%)	R@k-T1 (%)	R@k-T2 (%)	R@k-T3 (%)
5%	AE	**87.86±0.09**	35.85±0.10	42.77±0.32	45.52±0.19	76.30±0.52	**21.13±0.93**	39.13±0.36
	LSTMAE	85.60±0.09	26.11±0.12	36.76±0.15	34.54±0.30	59.68±0.40	16.14±0.40	27.80±0.98
	GCNAE	82.97±0.14	59.09±0.18	**56.35±0.25**	65.60±0.10	94.98±0.36	1.82±0.18	**100.00±0.00**
	TGCAE	85.34±0.33	**59.92±0.76**	56.68±0.41	**67.38±0.12**	**97.36±0.47**	4.78±0.28	100.00±0.00
10%	AE	**85.57±0.07**	43.54±0.19	52.37±0.22	52.08±0.15	88.99±0.35	**25.26±0.31**	42.00±0.30
	LSTMAE	82.91±0.13	34.34±0.13	44.84±0.25	43.85±0.08	81.62±0.44	18.75±0.41	31.19±0.47
	GCNAE	82.38±0.09	60.44±0.24	60.90±0.55	64.72±0.21	97.06±0.23	3.88±0.13	93.23±0.43
	TGCAE	84.47±0.14	**62.20±0.50**	**67.66±0.75**	**68.26±0.43**	**97.78±0.87**	9.83±0.48	**97.16±1.42**
15%	AE	**83.80±0.09**	48.69±0.19	53.66±0.30	55.88±0.18	93.20±0.27	**28.70±0.22**	45.73±0.25
	LSTMAE	81.06±0.09	40.40±0.15	45.33±0.18	48.69±0.15	89.17±0.27	21.95±0.21	34.94±0.21
	GCNAE	81.07±0.05	60.48±0.22	49.14±0.35	57.30±0.19	97.75±0.09	7.76±0.08	66.39±0.62
	TGCAE	83.67±0.15	**64.35±0.84**	**56.41±0.75**	**64.46±0.63**	**98.80±0.35**	15.38±0.76	**79.19±2.31**
20%	AE	**82.46±0.08**	53.12±0.21	50.54±0.34	58.63±0.17	95.13±0.36	**31.46±0.17**	49.31±0.31
	LSTMAE	79.76±0.08	45.54±0.14	41.16±0.23	52.47±0.19	92.56±0.35	25.55±0.21	39.31±0.30
	GCNAE	79.36±0.09	61.34±0.19	45.53±0.15	54.44±0.24	98.02±0.13	11.53±0.20	53.78±0.76
	TGCAE	82.45±0.34	**65.71±1.64**	**49.47±2.98**	**62.42±1.08**	**98.61±0.61**	19.73±1.10	**68.93±2.45**
30%	AE	**80.13±0.06**	60.04±0.13	36.93±0.34	**62.77±0.12**	95.97±0.23	**37.37±0.24**	54.97±0.21
	LSTMAE	OOM	OOM	OOM	OOM	OOM	OOM	OOM
	GCNAE	76.54±0.07	65.20±0.18	40.94±0.20	55.68±0.11	98.41±0.18	19.58±0.32	49.06±0.32
	TGCAE	79.57±0.27	**69.04±0.43**	**41.87±2.92**	62.41±0.67	**99.07±0.11**	26.24±0.94	**61.93±1.62**
40%	AE	**78.02±0.05**	65.74±0.08	25.85±0.22	66.41±0.04	96.58±0.25	**43.27±0.20**	59.37±0.10
	LSTMAE	OOM	OOM	OOM	OOM	OOM	OOM	OOM
	GCNAE	69.36±10.82	69.57±0.08	26.81±14.99	**67.96±17.91**	**99.00±0.56**	42.03±32.41	**62.85±20.77**
	TGCAE	76.63±0.21	**71.88±0.73**	**30.41±1.64**	63.85±0.32	98.09±1.05	32.38±0.72	61.09±2.20

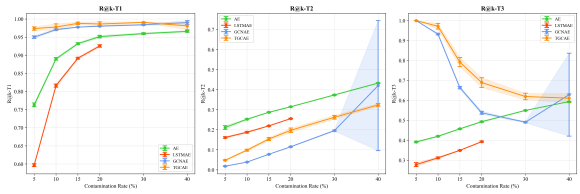

Fig. 3 Recall@k-Ti Performance Comparison Across Models for Dif-ferent Anomaly Types in the Real Dataset

timestamp anomaly detection—which represents the most challenging category for graph neural network-based anomaly detection. This improvement can be attributed to the temporal graph convolution mechanism, which effectively captures the inherent structural dependencies and temporal dynamics in business process sequences.

In summary, the experimental results on real-world datasets fully verify the effectiveness and superiority of the TGCAE model. Across all six contamination levels, TGCAE achieved optimal or suboptimal performance in key metrics such as $AUC-ROC$, $AUC-PR$, $F1-score$, and $Recall@k$. Moreover, TGCAE also showed a consistent trend of performance improvement in timestamp anomaly detection tasks, proving the unique advantage of the temporal graph convolution mechanism in capturing complex spatiotemporal dependencies. In addition, the stable performance and excellent memory efficiency of the model in high-contamination scenarios provide a solid theoretical and experimental foundation for its practical deployment in real-world business process anomaly detection.

5.3.2 Results on Synthetic Logs

Synthetic datasets can simulate anomaly patterns that are difficult to obtain or rare in real-world business processes, compensating for the deficiency of real-world datasets in anomaly type coverage. They are suitable for in-depth analysis of

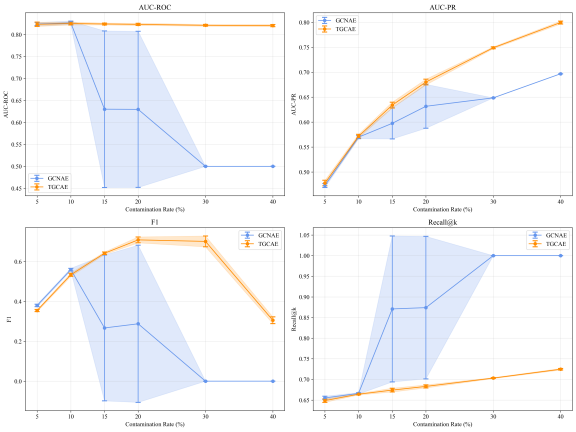

Fig. 4 Comparison of Main Performance Metrics Across Models in the Synthetic Dataset

the model's sensitivity and robustness to different data characteristics. Table 4 presents the performance of different autoencoder models under various data contamination levels in the DS2 dataset. It is worth noting that both the AE and LSTMAE models encountered out-of-memory issues under all contamination rate conditions, exposing their defects of excessive memory occupation and poor adaptability to computing resources when processing this dataset. This also directly restricts their practical application feasibility in this data scenario, while highlighting the significant advantages of graph structure-based methods in handling large-scale and complex business process data. However, the literature [8] has explored the superior performance of the GCNAE model over AE and LSTMAE on the DS2 dataset; therefore, on the synthetic dataset, we only conducted a systematic comparative analysis between TGCAE and GCNAE.

As shown in Table 4 and Fig. 4, under low contamination rate conditions, the performance of TGCAE and GCNAE was similar. However, with the increase of contamination rate, the advantages of TGCAE gradually became prominent.

At a contamination rate of 15%, GCNAE exhibited severe performance fluctuations with abnormally high standard deviations in various metrics, while TGCAE consistently maintained stable output performance. This stability advantage was further amplified at higher contamination rates: when the contamination rate was 20%,

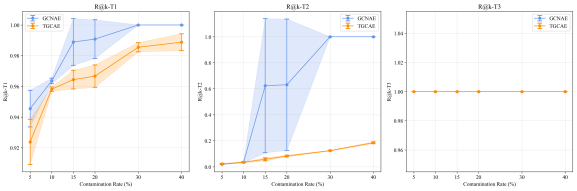

Fig. 5 Recall@k-Ti Performance Comparison Across Models for Dif-ferent Anomaly Types in the Synthetic Dataset

the $F1-score$ of TGCAE reached $70.88 \pm 1.48\%$, while that of GCNAE was only $28.78 \pm 39.41\%$.

It is particularly noteworthy that when the contamination rate exceeded 30%, GCNAE almost completely failed, which can be confirmed by the sharp collapse of its $AUC-ROC$ and $F1$ curves in Fig. 4. At this time, the Recall@k value was 100%, indicating that the model lost its judgment ability and regarded all cases as anomalies. In contrast, TGCAE maintained robust performance in all evaluation metrics, demonstrating its excellent noise tolerance and significant applicability in real-world scenarios with high data contamination.

Combined with Table 4 and Fig. 5, through the detection and analysis of different anomaly types, the DS2 dataset presented distinct characteristics in various anomaly scenarios.

In the tasks of attribute anomaly and activity anomaly detection, both TGCAE and GCNAE achieved extremely high recall rates, indicating that these two types of anomalies have highly distinguishable patterns that can be easily captured by graph-based models.

In contrast, timestamp anomaly detection was more challenging and its performance decreased significantly with increasing contamination rate. Although the recall rate of TGCAE in the timestamp anomaly showed a steady upward trend, reflecting its adaptive learning ability in high-noise environments, its overall performance was still inferior to that of GCNAE. This difference is mainly due to the insufficient temporal dependency information embedded in the DS2 dataset itself, which limits the effective utilization of temporal information by TGCAE; while GCNAE is less dependent on temporal features in its design, so it is less affected by this dataset limitation.

In conclusion, the experimental results in the DS2 dataset verify the excellent robustness and

Table 4 Comparative Analysis of Model Metrics on Synthetic Dataset

Contam. Rate(%)	Model	AUC-ROC (%)	AUC-PR (%)	F1 (%)	Recall@k (%)	R@k-T1 (%)	R@k-T2 (%)	R@k-T3 (%)
5%	GCNAE	**82.40±0.35**	47.28±0.39	**37.98±0.59**	**65.52±0.42**	**94.55±1.19**	2.01±0.24	**100.00±0.00**
	TGCAE	82.33±0.49	**47.78±0.55**	35.43±0.52	64.87±0.42	92.38±1.47	2.22±0.48	**100.00±0.00**
10%	GCNAE	**82.46±0.34**	56.99±0.35	**56.15±0.96**	**66.68±0.14**	**96.39±0.19**	**3.64±0.45**	**100.00±0.00**
	TGCAE	82.51±0.33	**57.17±0.37**	53.37±0.82	66.42±0.17	95.82±0.14	3.43±0.39	**100.00±0.00**
15%	GCNAE	63.02±17.83	59.74±3.10	26.55±36.36	**87.06±17.71**	**98.86±1.57**	**62.33±51.58**	**100.00±0.00**
	TGCAE	**82.40±0.22**	**63.41±0.58**	**64.17±0.62**	67.42±0.46	96.44±0.61	5.81±1.19	**100.00±0.00**
20%	GCNAE	62.97±17.76	63.17±4.40	28.78±39.41	**87.40±17.26**	**99.08±1.26**	**63.11±50.52**	**100.00±0.00**
	TGCAE	**82.31±0.25**	**68.03±0.57**	**70.88±1.48**	68.31±0.37	96.67±0.73	8.28±0.66	**100.00±0.00**
30%	GCNAE	50.00±0.00	64.86±0.00	0.00±0.00	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**
	TGCAE	**82.10±0.20**	**74.90±0.19**	**70.12±2.73**	70.33±0.12	98.55±0.30	12.44±0.32	**100.00±0.00**
40%	GCNAE	50.00±0.00	69.69±0.00	0.00±0.00	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**
	TGCAE	**82.03±0.24**	**79.97±0.29**	**30.55±1.69**	72.47±0.19	98.88±0.55	18.52±0.76	**100.00±0.00**

scalability of TGCAE in large-scale and high-contamination scenarios. The temporal graph convolution mechanism not only solves the memory bottleneck problem of traditional methods, but also, more importantly, maintains stable detection performance under extreme noise conditions, which has important practical significance for the application of real-world business process anomaly detection.

5.3.3 Overall Performance Comparison

Fig. 6 presents a radar chart for the comprehensive comparison of four anomaly detection models across seven key performance metrics in the BPIC 2017 and DS2 datasets. It can be seen from the figure that the TGCAE model exhibited optimal or near-optimal performance in most metrics, especially significantly outperforming other baseline models in the evaluation dimensions of $AUC-PR$, $F1-score$, and $Recall@k$, which verifies its superiority in the anomaly detection task of complex process logs.

It should be noted that although GCNAE achieved the optimal recall rate in the DS2 dataset, this was actually a false performance improvement. The root cause is that the model completely failed under some high contamination rates, leading to extensive misjudgment in the model's determination of abnormal samples, thereby artificially inflating the recall rate. In contrast, the performance of the traditional AE model

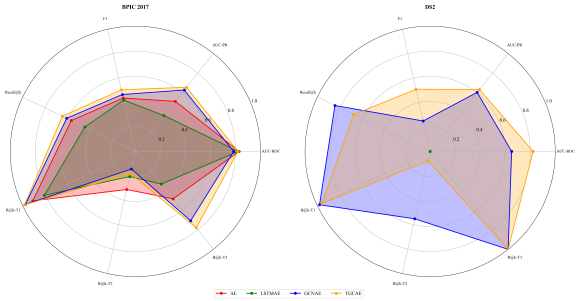

Fig. 6 Multi-dimensional Comparison Radar Chart of Model Perfor-mance

was more conservative, with all metrics consistently remaining in a stable range without extreme fluctuations; while LSTMAE and GCNAE showed significant performance oscillations in some key metrics. This phenomenon fully exposes the inherent limitations of sequence-dependent modeling or pure graph structure modeling in complex anomaly detection scenarios.

In this study, we systematically evaluated the statistically significant differences between the proposed TGCAE model and the baseline models across multiple performance metrics. By adopting two complementary statistical methods—paired t-test and Wilcoxon/Mann-Whitney U test—we aimed to comprehensively verify whether the superiority of the TGCAE model in different evaluation dimensions is statistically significant rather than accidental results.

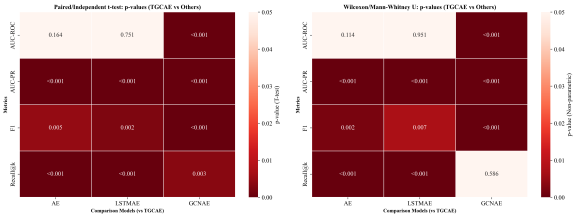

Fig. 7 Heatmap of p-values from statistical significance tests

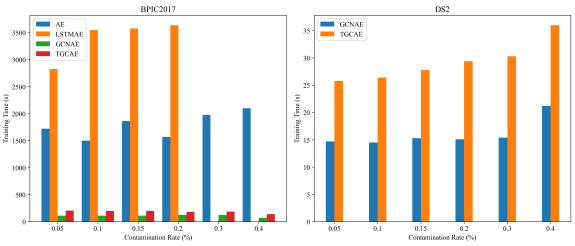

Fig. 8 Fig.8 Training Time Comparison

As can be seen from the heat map in Fig.7, TGCAE exhibited significantly better performance than the baseline models in most metrics. Especially in the key metrics of AUC-PR and F1-score, the p-values with all comparison models were less than 0.001, indicating that its performance advantage is highly statistically significant. This finding strongly supports the effectiveness of the TGCAE model in capturing complex time series patterns and identifying rare abnormal events. It should be noted that, with respect to the $AUC-ROC$ metric, the performance differences between TGCAE and both AE and LSTMAE did not reach statistical significance; however, the difference between TGCAE and GCNAE—another approach based on graph neural networks, approach—reached a high level of statistical significance. The main reason why the difference with GCNAE in the $Recall@k$ metric did not reach a statistically significant level is that GCNAE failed under high contamination rates in the DS2 dataset, leading to the artificial inflation of its recall rate. The results of the two test methods are highly consistent, further enhancing the reliability of the conclusions.

Finally, we evaluated the training time of each model in the two datasets, and the results are shown in Fig. 8.

In the BPIC 2017 dataset, there were significant differences in the training efficiency of different models. LSTMAE had the longest training time, followed by AE. In contrast, the graph structure-based GCNAE and TGCAE had significantly higher training efficiency. This result highlights the high efficiency of the graph neural network architecture in processing object-centric event logs, which avoids the high computational overhead caused by the sequence-based LSTMAE and the traditional autoencoder AE.

In the DS2 dataset, both GCNAE and TGCAE demonstrated high training efficiency. Although TGCAE incurred a certain amount of additional computational overhead compared to GCNAE due to the introduction of modules such as the temporal encoder and the adaptive gating fusion mechanism, this overhead brought about a comprehensive leap in model performance. In the core anomaly detection metrics, TGCAE achieved significant improvements, and its performance gain far exceeded the slight increase in computational cost.

In summary, the results indicate that while maintaining training efficiency, TGCAE achieved a better balance between performance and efficiency through the adaptive fusion of temporal information and graph structure features, making it more suitable for practical scenarios with high training efficiency requirements.

5.3.4 Ablation Experiments

To fully understand the contribution of each core component in the TGCAE framework to anomaly detection performance, three systematic ablation experiments were designed in this study: removing the temporal encoder (TGCAE-NoTime), removing the gating fusion mechanism (TGCAE-NoGate) and adopting fixed fusion weights instead of dynamically learned weights (TGCAE-FixedW). The experimental results are presented in the form of Performance Drop (s%), which is defined as Eq. (23).

$$\Delta P = \frac{P_{full} - P_{variant}}{P_{full}} \times 100\% \qquad (23)$$

where P_{full} is the performance mean of the complete model, and $P_{variant}$ is the performance mean of the variant model. A positive value indicates that the complete model performs better.

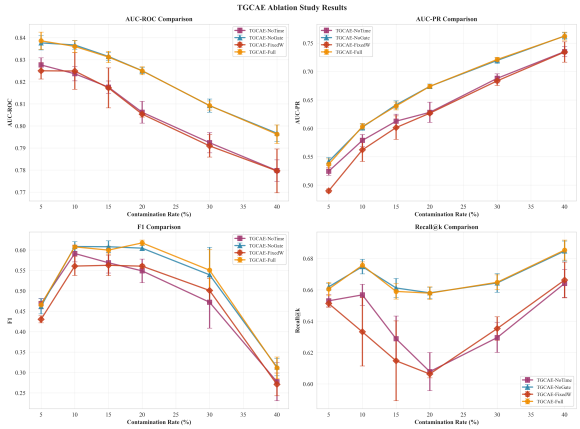

Fig. 9 Ablation Study: Impact of Removing Key Components on Model Performance

Fig. 10 Contribution Analysis of Each Component to Model Performance

The results are shown in Figs. 9 and 10. From the overall trend, the three ablation variants exhibit significant differences in performance under different contamination rates, revealing the unique role of each component in the model.

(1) TGCAE-NoTime: By removing the temporal encoder and retaining only the structural encoder, the impact of temporal information on anomaly detection performance was evaluated. TGCAE-NoTime showed varying degrees of performance degradation at all contamination rates, especially more obvious degradation at high contamination rates, indicating that temporal information is crucial to overall discriminative ability and comprehensive performance of the model. This verifies the necessity of introducing the temporal encoder in our architecture design—even when static structural features are sufficiently strong, information in the temporal dimension can still provide key supplementary signals, helping the model identify complex time-sensitive anomalies more accurately.

(2) TGCAE-NoGate: By removing the adaptive gating network and adopting direct concatenation or other static fusion strategies, the role of the gating mechanism in multi-modal information fusion was evaluated. In most contamination rates, degradation of performance was minimal and even a slight improvement occurred in some cases. However, this does not mean that the gating mechanism is redundant; in extreme noise environments or specific data distributions, the dynamic adjustment capability of the gating mechanism may play a key role. This "apparent similarity" precisely reflects the role of the gating mechanism in the robustness and adaptive capability of the model.

(3) TGCAE-FixedW: By using preset fixed weights instead of dynamically learned gating weights, it was verified whether the adaptive mechanism truly improves model performance. This variant showed significant performance degradation at all contamination rates. This result strongly supports the importance of the adaptive weight mechanism. By dynamically learning the optimal fusion weight for each node, the model can adaptively adjust the contribution of temporal information according to the characteristics of the data, thus maintaining high performance in complex scenarios. Although fixed weights simplify the model structure, they sacrifice this flexibility, leading to a significant decline in performance when facing diverse anomaly patterns.

In summary, the results of the ablation experiments verify the rationality of the design of each core component in the TGCAE framework. As a basic module of the model, the indispensability of the temporal encoder has been fully confirmed; although the gating fusion mechanism showed stable performance in the current experiments, its potential dynamic adjustment capability is worthy of further exploration; the adaptive weight mechanism has also been proven to be the key to improving model performance, and its absence will lead to significant degradation of the model in multiple metrics.

6 Conclusion

Anomaly detection in business process event logs plays a crucial role in ensuring operational compliance and identifying process inefficiencies, enabling enterprises to maintain service quality and reduce risks in complex environments. Object-centric event logs provide a more comprehensive representation of real-world business processes by capturing multiple interacting objects and their relationships, overcoming the limitation of traditional case-centric logs that flatten multi-dimensional processes into linear sequences. This study proposes a novel Temporal Graph Convolutional AutoEncoder (TGCAE) for anomaly detection in object-centric event logs, addressing the key limitation of existing graph-based methods that ignore temporal dependencies in business process mining. This method does not require predefined process models, prior knowledge of contamination rates, or clean training sets, expanding its scope of application. Meanwhile, the model exhibits strong adaptability under high-noise conditions and, combined with superior memory efficiency, enables it to perform anomaly detection in real-world business processes.

Future research directions can explore Transformer or multi-dimensional temporal convolutional network architectures to capture long-range temporal dependencies and periodic patterns in event sequences, further improving the detection capability for complex temporal anomalies and making up for the shortcomings of TGCAE. Future work can also integrate temporal knowledge graph components to model the temporal changes of process topology and the evolution of knowledge associations, enabling the model to adapt to both attribute anomalies and structural drift, accurately capture knowledge association anomalies in the temporal dimension, and further enhance the detection accuracy and generalization ability of temporal anomalies.

Acknowledgements. The authors also gratefully acknowledge the helpful comments and suggestions of the reviewers, which have improved the presentation.

Author contributions. J.C. completed the research design, conducted data analysis, and wrote the main manuscript. H.F. provided guidance on the paper. C. L. objectively reviewed the article. All authors read and approved the final manuscript.

Funding. Supported by the National Key R&D Program, China (No.2023YFC3807501), and National funds through FCT (Fundação para a Ciência e a Tecnologia), under the project - UIDB/04152 - Centro de Investigação em Gestão de Informação (MagIC)/NOVA IMS.

Data availability. Data supporting the findings of this study are available from the corresponding author, Huan Fang, upon reasonable request.

Declarations

Conflict of interest. No potential conflict of interest was reported by the authors.

References

[1] Y. Zhou, "Application of anomaly detection mechanism in large-scale data processing," *European Journal of AI, Computing & Informatics*, vol. 1, no. 1, pp. 78–84, 2025.

[2] T. Ouyang and X. Zhang, "Fuzzy rule-based anomaly detectors construction via information granulation," *Information Sciences*, vol. 622, pp. 985–998, 2023.

[3] R. Sarno, F. Sinaga, and K. R. Sungkono, "Anomaly detection in business processes using process mining and fuzzy association rule learning," *Journal of Big Data*, vol. 7, no. 1, p. 5, 2020.

[4] P. J. Rousseeuw and M. Hubert, "Anomaly detection by robust statistics," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 8, no. 2, p. e1236, 2018.

[5] H. Kim, B. S. Lee, W.-Y. Shin, and S. Lim, "Graph anomaly detection with graph neural networks: Current status and challenges," *IEEe Access*, vol. 10, pp. 111820–111829, 2022.

[6] Z. Yuan, Q. Sun, H. Zhou, M. Shao, and X. Fu, "A comprehensive survey on gnn-based anomaly detection: taxonomy, methods, and the role of large language models," *International Journal of Machine Learning and Cybernetics*, pp. 1–26, 2025.

[7] H. Kim, B. S. Lee, W.-Y. Shin, and S. Lim, "Graph anomaly detection with graph neural networks: Current status and challenges," *IEEe Access*, vol. 10, pp. 111820–111829, 2022.

[8] A. Niro and M. Werner, "Detecting anomalous events in object-centric business processes via graph neural networks," in *International Conference on Process Mining*, pp. 179–190, Springer, 2023.

[9] W. Guan, J. Cao, Y. Gu, and S. Qian, "Gama: A multi-graph-based anomaly detection framework for business processes via graph neural networks," *Information Systems*, vol. 124, p. 102405, 2024.

[10] Z. Tian, M. Zhuo, L. Liu, J. Chen, and S. Zhou, "Anomaly detection using spatial and temporal information in multivariate time series," *Scientific Reports*, vol. 13, no. 1, p. 4400, 2023.

[11] H. T. C. Nguyen, S. Lee, J. Kim, J. Ko, and M. Comuzzi, "Autoencoders for improving quality of process event logs," *Expert Systems with Applications*, vol. 131, pp. 132–147, 2019.

[12] P. Krajsic and B. Franczyk, "Variational autoencoder for anomaly detection in event data in online process mining.," in *ICEIS (1)*, pp. 567–574, 2021.

[13] T. Nolle, S. Luettgen, A. Seeliger, and M. Mühlhäuser, "Binet: Multi-perspective business process anomaly classification," *Information Systems*, vol. 103, p. 101458, 2022.

[14] J. Lahann, P. Pfeiffer, and P. Fettke, "Lstm-based anomaly detection of process instances: benchmark and tweaks," in *International Conference on Process Mining*, pp. 229–241, Springer, 2022.

[15] L. Ke, F. Huan, X. Yifei, and S. Chifeng, "Multi-task prediction method based on ggcn for object centric event logs," *IEEE Access*, 2025.

[16] W. M. van der Aalst, "Object-centric process mining: dealing with divergence and convergence in event data," in *International Conference on Software Engineering and Formal Methods*, pp. 3–25, Springer, 2019.

[17] B. Kang, D. Kim, and S.-H. Kang, "Real-time business process monitoring method for prediction of abnormal termination using knni-based lof prediction," *Expert Systems with Applications*, vol. 39, no. 5, pp. 6061–6068, 2012.

[18] A. Sureka, "Kernel based sequential data anomaly detection in business process event logs," *arXiv preprint arXiv:1507.01168*, 2015.

[19] F. Folino, G. Greco, A. Guzzo, and L. Pontieri, "Mining usage scenarios in business processes: Outlier-aware discovery and run-time prediction," *Data & Knowledge Engineering*, vol. 70, no. 12, pp. 1005–1029, 2011.

[20] G. M. Tavares and S. Barbon Jr, "Analysis of language inspired trace representation for anomaly detection," in *International Conference on Theory and Practice of Digital Libraries*, pp. 296–308, Springer, 2020.

[21] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.

[22] S. B. Junior, P. Ceravolo, E. Damiani, N. J. Omori, and G. M. Tavares, "Anomaly detection on event logs with a scarcity of labels," in *2020 2nd International Conference on Process Mining (ICPM)*, pp. 161–168, IEEE, 2020.

[23] W. Guan, J. Cao, Y. Gu, and S. Qian, "Grasped: A gru-ae network based multi-perspective business process anomaly detection model," *IEEE Transactions on Services Computing*, vol. 16, no. 5, pp. 3412–3424, 2023.

[24] T. Nolle, S. Luettgen, A. Seeliger, and M. Mühlhäuser, "Analyzing business process anomalies using autoencoders," *Machine Learning*, vol. 107, no. 11, pp. 1875–1893, 2018.

[25] T. Nolle, A. Seeliger, and M. Mühlhäuser, "Unsupervised anomaly detection in noisy business process event logs using denoising autoencoders," in *International conference on discovery science*, pp. 442–456, Springer, 2016.

[26] J. Evermann, J.-R. Rehse, and P. Fettke, "Predicting process behaviour using deep learning," *Decision Support Systems*, vol. 100, pp. 129–140, 2017.

[27] N. Tax, I. Verenich, M. La Rosa, and M. Dumas, "Predictive business process monitoring with lstm neural networks," in *International conference on advanced information systems engineering*, pp. 477–492, Springer, 2017.

[28] A. Graves, "Long short-term memory," *Supervised sequence labelling with recurrent neural networks*, pp. 37–45, 2012.

[29] T. Nolle, A. Seeliger, and M. Mühlhäuser, "Binet: multivariate business process anomaly detection using deep learning," in *International Conference on Business Process Management*, pp. 271–287, Springer, 2018.

[30] M. Ebrahim and S. A. H. Golpayegani, "Anomaly detection in business processes logs using social network analysis," *Journal of Computer Virology and Hacking Techniques*, vol. 18, no. 2, pp. 127–139, 2022.

[31] S. J. Leemans, D. Fahland, and W. M. Van der Aalst, "Scalable process discovery and conformance checking," *Software & Systems Modeling*, vol. 17, no. 2, pp. 599–631, 2018.

[32] F. Vitale, M. Pegoraro, W. M. van der Aalst, and N. Mazzocca, "Control-flow anomaly detection by process mining-based feature extraction and dimensionality reduction," *Knowledge-Based Systems*, vol. 310, p. 112970, 2025.

[33] K. Böhmer and S. Rinderle-Ma, "Multi-perspective anomaly detection in business process execution events," in *OTM Confederated International Conferences" On the Move to Meaningful Internet Systems"*, pp. 80–98, Springer, 2016.

[34] D. Rahmawati, R. Sarno, C. Fatichah, and D. Sunaryono, "Fraud detection on event log of bank financial credit business process using hidden markov model algorithm," in *2017 3rd International Conference on Science in Information Technology (ICSITech)*, pp. 35–40, IEEE, 2017.

[35] C. Linn and D. Werth, "Sequential anomaly detection techniques in business processes," in *International conference on business information systems*, pp. 196–208, Springer, 2016.

[36] S. Huo, H. Völzer, P. Reddy, P. Agarwal, V. Isahagian, and V. Muthusamy, "Graph autoencoders for business process anomaly detection," in *International Conference on Business Process Management*, pp. 417–433, Springer, 2021.

[37] W. Guan, J. Cao, J. Gao, H. Zhao, and S. Qian, "Dabl: Detecting semantic anomalies in business processes using large language models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, pp. 11735–11744, 2025.

[38] A. Berti, I. Koren, J. N. Adams, G. Park, B. Knopp, N. Graves, M. Rafiei, L. Liß, L. T. G. Unterberg, Y. Zhang, *et al.*, "Ocel (object-centric event log) 2.0 specification," *arXiv preprint arXiv:2403.01975*, 2024.

[39] A. Goossens, J. De Smedt, and J. Vanthienen, "Object-centric event logs: specifications, comparative analysis and refinement," *arXiv preprint arXiv:2405.12709*, 2024.

[40] A. Berti, U. Jessen, W. M. van der Aalst, and D. Fahland, "Challenges of anomaly detection in the object-centric setting: Dimensions and the role of domain knowledge," *arXiv preprint arXiv:2407.09023*, 2024.

[41] J. N. Adams, D. Schuster, S. Schmitz, G. Schuh, and W. M. Van Der Aalst, "Defining cases and variants for object-centric event data," in *2022 4th International Conference on Process Mining (ICPM)*, pp. 128–135, IEEE, 2022.

[42] O. O. Amin, W. M. Abdelmoez, and M. Shaheen, "Anomaly detection of object-centric event logs using event knowledge graph," in *2024 34th International Conference on Computer Theory and Applications (ICCTA)*, pp. 349–355, IEEE, 2024.

[43] Y. Zhou, "Optimization of multi dimensional time series data anomaly detection model based on graph deviation network and convolutional neural network," *Procedia Computer Science*, vol. 261, pp. 199–206, 2025.

Jun Cao received the B.S degree in Data Science and Big Data Technology from Anhui University of Science and Technology, China, in 2024. Currently, He is a postgraduate student at the college of Mathematics and Big data in Anhui University of Science and Technology. His research interest includes process mining, and predictive process monitoring.

Huan Fang received the B.S. degree in Information and Computing from Anhui University of Science and Technology, China, in 2003 and M.S. degree in Computer Science and Engineering from Shandong University of Science and Technology, China, in 2006 and Ph.D. degree in Computer Science and Engineering from Hefei University of Technology, China, in 2013. From 2018 to 2021, she was a postdoc in Anhui University of Science and Technology, Huainan, China. Her research interests include Petri nets, process mining, change mining, and intelligent control. She is a professor of the School of Mathematics and Big Data in Anhui University of Science and Technology.

Cong Liu received the B.S. and the M.S. degree in computer software and theory from Shandong University of Science and Technology, Qingdao, China, in 2013 and 2015 respectively. He received the Ph.D. degree in the Department of Mathematics and Computer Science, Eindhoven University of Technology, 2019. He is an invited full professor in the NOVA Information Management School, Nova University of Lisbon. His research interests are in the areas of process mining, business process management, and artificial intelligence.